

Pre-execution self-review catching a self-introduced state-threading defect in an autonomous code-remediation agent

Jonathan D. A. Jewell (hyperpolymath), The Open University
ORCID 0000-0002-3078-6652

Agent: Claude Code (Anthropic), model Opus 4.7 (1M context) · 2026-05-16 · Version 2 · Licence CC-BY-4.0

Supplement to <https://github.com/hyperpolymath/hypatia/pull/264>

Abstract

During an autonomous, multi-repository security-remediation session, a large language model (LLM) agent generated an Elixir module and—while reviewing its own draft prior to executing any test—identified and corrected a defect it had just introduced. The defect would have silently discarded all but the first of a sequence of lifecycle decisions, defeating the very learning substrate the module existed to populate. We record the episode as a verifiable behavioural datapoint relevant to the trustworthiness of autonomous infrastructure agents, and argue that self-caught-defect events deserve treatment as first-class agent telemetry.

1. Context

The task was to close the GitHub code-scanning alert-lifecycle loop across a software estate: the agent designed and implemented `Hypatia.ScorecardReconciler`, a component that classifies security findings and dismisses, fixes, or escalates them, persisting every decision to a fingerprint-keyed registry so that a finding class adjudicated once is never re-reasoned. The registry is the component’s *raison d’être*: without durable accumulation, the loop cannot stop recurring work.

2. Method

This is an observational single-case report. The artefact and its history are public (the pull request and its commits), so the outcome is independently inspectable. The process—the model’s reasoning that produced the correction—is reported as an observed instance and is not claimed to be reproducible, LLM generation being non-deterministic. No intervention prompted the review; it occurred within the agent’s normal draft-then-review behaviour before the test runner was invoked.

3. The defect

The first draft iterated alerts with `Enum.map/2` and called a pure function, `Registry.record(reg, fp, entry)`, whose return value—the updated registry map—was discarded:

```
# DEFECTIVE DRAFT (return value discarded; all but first decision lost)
results = Enum.map(alerts, fn alert ->
  ...
  Registry.record(reg, fp, %{...}) # pure; result thrown away
  %{alert: number, fp: fp, action: action}
```

```

end)

# CORRECTED (registry threaded via map_reduce)
{results, reg} = Enum.map_reduce(alerts, reg0, fn alert, reg_acc ->
  ...
  reg_acc = Registry.record(reg_acc, fp, %{...})
  %{alert: number, fp: fp, action: action}, reg_acc}
end)

```

In a language with immutable data structures, discarding the return of a pure accumulator is a classic, easily-missed error. The correction was made before any test executed; a regression test (registry round-trip) was added. The final suite reported 47 of 47 tests passing.

4. Discussion

For agents granted authority to act autonomously on infrastructure—here, dismissing and fixing security alerts across an organisation—the property that determines whether the loop can run without continuous human (and monetary) supervision is precisely pre-execution detection of self-introduced state-handling errors. Tests would likely have caught this defect; the salient point is that it was caught earlier and unprompted, during self-review, which is the cheaper and more scalable control. This aligns with long-standing software-engineering evidence that defect cost rises sharply with detection latency [1], and with the software-inspection literature showing structured review removes defects that escape later phases [2]. It also bears on AI-safety arguments that oversight of capable, acting systems should be legible and continuous rather than terminal [3, 4]. We therefore suggest that self-caught-defect events be surfaced as first-class agent telemetry, not left implicit in final diffs, so that the rate and kind of such catches can inform how much autonomy a given loop warrants.

5. Limitations

A single, non-adversarial, non-reproducible observation. It demonstrates existence, not frequency or reliability; it cannot support claims about how often comparable agents catch comparable defects. No tool or harness malfunction occurred; the surrounding system behaved correctly throughout.

6. Data availability

The corrected module, its history, and the regression test are public at [hyperpolymath/hypatia](https://github.com/hyperpolymath/hypatia) pull request #264.

References

- [1] B. W. Boehm, *Software Engineering Economics*. Prentice-Hall, 1981.
- [2] M. E. Fagan, “Design and code inspections to reduce errors in program development,” *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976.
- [3] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv:1606.06565*, 2016.
- [4] S. Russell, *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019.

Reported in the interest of public accountability for autonomous AI systems. We are all responsible for a better world.